



Unidad 2 – Práctica Evaluación Continua 2

Instrucciones:

- Una vez finalizada, entrega la actividad en formato PDF en la tarea de Aules. Recuerda que los enunciados también deben aparecer en el PDF.
- Al final del ejercicio se encuentra la rúbrica de evaluación

RA's:

- RA1 - Implanta arquitecturas web analizando y aplicando criterios de funcionalidad
- RA2 - Implanta aplicaciones web en servidores web, evaluando y aplicando criterios de configuración para su funcionamiento seguro.

1. (4 pts) Clona el siguiente repositorio:

<https://github.com/JuanraCollado/octopus-underwater-app>.

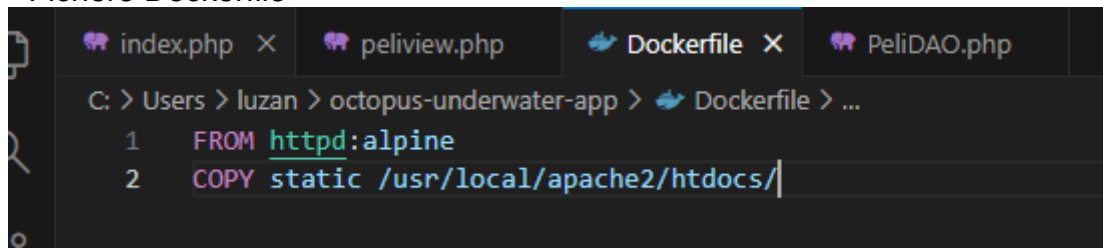
Modifica el fichero Dockerfile para que la imagen esté basada en un servidor Apache sobre SO Alpine.

Al igual que hicimos con nginx, forzaremos que el Dockerfile realice la copia de los ficheros de la carpeta static a la carpeta principal del servidor Apache.

Una vez todo configurado, crearemos y arrancaremos la aplicación mapeando el puerto 80 del contenedor a nuestro puerto 9090.

Añade capturas de:

- Fichero Dockerfile



```
C: > Users > luzan > octopus-underwater-app > Dockerfile > ...
1 FROM httpd:alpine
2 COPY static /usr/local/apache2/htdocs/
```

- Creación del contenedor

```

C:\Users\luzan\octopus-underwater-app>docker build -t octopus .
[+] Building 12.0s (8/8) FINISHED                                docker:desktop-linux
=> [internal] load build definition from Dockerfile               0.1s
=> => transferring dockerfile: 94B                                0.1s
=> [internal] load metadata for docker.io/library/httpd:alpine   2.1s
=> [auth] library/httpd:pull token for registry-1.docker.io      0.0s
=> [internal] load .dockerignore                                  0.0s
=> => transferring context: 2B                                     0.0s
=> [internal] load build context                                  0.0s
=> => transferring context: 900B                                    0.0s
=> [1/2] FROM docker.io/library/httpd:alpine@sha256:07b2fabb7029a0b8aeb2e0fd02651c28fe22c21c5b5a59d6ff5b022791fc 8.1s
=> => resolve docker.io/library/httpd:alpine@sha256:07b2fabb7029a0b8aeb2e0fd02651c28fe22c21c5b5a59d6ff5b022791fc 0.1s
=> => sha256:932e71301ea975508d2b04ea7713a9cc025f7e2f8430b87173515cf1f9f25a22 287B / 287B 0.4s
=> => sha256:843a2f73d27987f41be2716de7109d3da733e67f6b765bd1a509d9488eb33a4d 10.60MB / 10.60MB 4.9s
=> => sha256:efa28cc119fe44e224de55f89cd24f440c60de44c565423a017dff218db13796 5.76MB / 5.76MB 3.3s
=> => sha256:50fa142ca9bcb1a52dd6cc7c8cd19375c3d7c168fa5751a6b5bfc8fb8c1c7f43 146B / 146B 0.6s
=> => sha256:e5a1637a5ea0e57b68532eb2c59e9c00aa5b73e6d21bc1f2e486b5080592cc46 934B / 934B 0.5s
=> => extracting sha256:e5a1637a5ea0e57b68532eb2c59e9c00aa5b73e6d21bc1f2e486b5080592cc46 0.4s
=> => extracting sha256:50fa142ca9bcb1a52dd6cc7c8cd19375c3d7c168fa5751a6b5bfc8fb8c1c7f43 0.2s
=> => extracting sha256:4f4fb700ef54461cfa02571ae0db9a0dc1e0cbb5577484a6d75e68dc38eacc1 0.2s
=> => extracting sha256:843a2f73d27987f41be2716de7109d3da733e67f6b765bd1a509d9488eb33a4d 2.4s
=> => extracting sha256:efa28cc119fe44e224de55f89cd24f440c60de44c565423a017dff218db13796 0.5s
=> => extracting sha256:932e71301ea975508d2b04ea7713a9cc025f7e2f8430b87173515cf1f9f25a22 0.0s
=> [2/2] COPY static /usr/local/apache2/htdocs/                  0.4s
=> => exporting to image                                           0.7s
=> => exporting layers                                             0.2s
=> => exporting manifest sha256:8d1ea4b3d5ec4357ac4dcab91c0784cce15dfd4254ab3113be1e944950c42901 0.0s
=> => exporting config sha256:808af7c812243f3ffdc337d846c94d6ed0bfc05f3808f36dcb1178734824ded 0.0s
=> => exporting attestation manifest sha256:249c33e73bca9499477ded504baca30c4f7d135486d7f604954a26953b4688f4 0.1s
=> => exporting manifest list sha256:dcddc98a6e8ef26741c3d334e4d9d5897dcddce8fe2ed0aad0361dd2a9c7911e 0.0s
=> => naming to docker.io/library/octopus:latest                  0.0s
=> => unpacking to docker.io/library/octopus:latest               0.1s

```

- Contenedor corriendo (docker ps y ejecutándose en navegador)

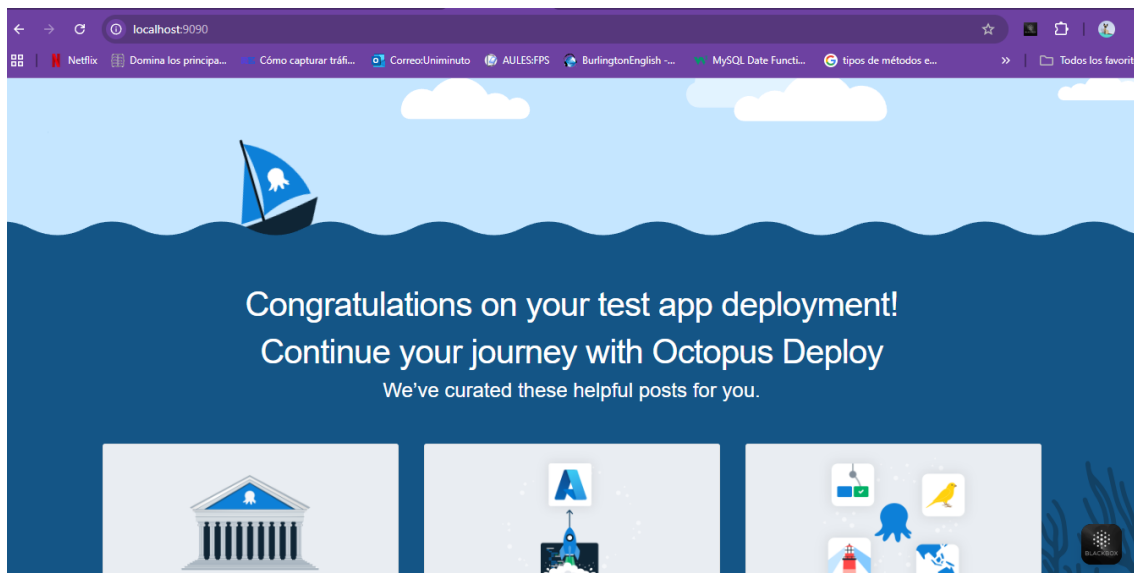
```

C:\Users\luzan\octopus-underwater-app>docker run -d -p 9090:80 octopus
8d114e947440616d5101b7caa2d40a368d2aac58b690cfd0e1c5a4409a9ca7e

C:\Users\luzan\octopus-underwater-app>docker ps
CONTAINER ID   IMAGE      COMMAND                  CREATED        STATUS        PORTS
AMES          8d114e947440   octopus   "httpd-foreground"     8 seconds ago   Up 7 seconds   0.0.0.0:9090->80/tcp, [::]:9090->80/tcp
harming_dhawan

C:\Users\luzan\octopus-underwater-app>

```



2. (6 pts) Vas a montar un pequeño entorno web en Docker compuesto por dos contenedores (lee hasta el final antes de empezar):

- Un contenedor con **MySQL** que guarde sus datos en un volumen persistente.

- Un contenedor con **phpMyAdmin** que te permita administrar la base de datos desde el navegador.
 - Puedes utilizar las siguientes imágenes o buscar las tu por tu cuenta: mysql:5.7 y phpmyadmin/phpmyadmin
- Hay que cumplir las siguientes condiciones:
- Deberás crear tu red Docker.

```
C:\Users\luzan\Desktop>docker network create redweb
800de094b9e093204ceec4010ec61385e4060c8e69b863b00197c8686a5cc452
```

- phpMyAdmin debe estar accesible en el puerto 8080 del host.
- MySQL debe utilizar un volumen con nombre para almacenar sus datos, de forma que la información no se pierda al borrar el contenedor.

```
C:\Users\luzan\Desktop>docker volume create mysql-data
mysql-data
```

- Accede desde el navegador a phpmyadmin (localhost:8080) y comprueba la conectividad con la base de datos creando una tabla.

Añade las siguientes capturas:

- Creación de ambos contenedores

```
C:\Users\luzan\Desktop>docker run -d --name mysql-container --network redweb -v mysql-data:/var/lib/mysql -e MYSQL_ROOT_PASSWORD=admin -e MYSQL_DATABASE=prueba mysql:5.7
Unable to find image 'mysql:5.7' locally
5.7: Pulling from library/mysql
6b95a940e7b6: Pull complete
20e4dcae4c69: Pull complete
43d05e938198: Pull complete
df9a4d85569b: Pull complete
064b2d298fba: Pull complete
e9f03a1c24ce: Pull complete
68c3898c2015: Pull complete
1c56c3d4ce74: Pull complete
90986bb8de6e: Pull complete
ae71319cb779: Pull complete
ffc89e9dfd88: Pull complete
Digest: sha256:4bc6bc963e6d8443453676cae56536f4b8156d78bae03c0145cbe47c2aad73bb
Status: Downloaded newer image for mysql:5.7
28abe0fb4c54bc25fed0bcf34de8010a2c7061552da565e2f3ca0fef610fa613
```

```
C:\Users\luzan\Desktop>docker run -d --name phpmyadmin --network redweb -e PMA_HOST=mysql-server -p 8080:80 phpmyadmin/phpmyadmin
5aaef3fa9cba1324d48f632a18b0a9b90e43baa8d9877afd1c3f8dd00ebe603a
```

- Ejecución de docker ps con ambos contenedores lanzados.

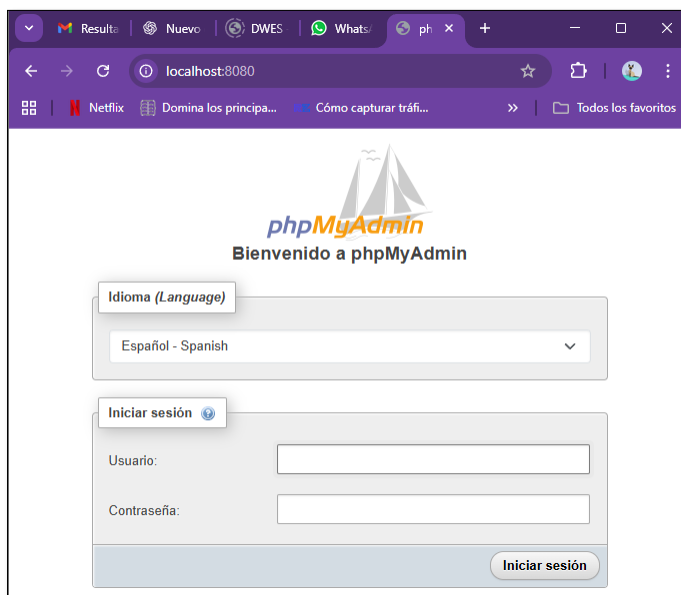
```
C:\Users\luzan\Desktop>docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED
STATUS        PORTS         NAMES
26f2f74975c9   mysql:5.7     "docker-entrypoint.s..." 6 second
s ago        Up 6 seconds  3306/tcp, 33060/tcp      mysql
- container
135daba7f5e2   phpmyadmin/phpmyadmin  "/docker-entrypoint...." 17 minut
es ago      Up 17 minutes  0.0.0.0:8080->80/tcp, [::]:8080->80/tcp  phpmy
admin
```

- Comprobación de conectividad mediante exec

```
C:\Users\luzan\Desktop>docker exec -it 5aaef3fa9cba /bin/bash
root@5aaef3fa9cba:/var/www/html# hostname
5aaef3fa9cba

root@5aaef3fa9cba:/var/www/html# printenv | grep PMA
PMA_SSL_DIR=/etc/phpmyadmin/ssl
PMA_HOST=mysql-server
root@5aaef3fa9cba:/var/www/html# |
```

- Comprobación de conectividad mediante acceso por web



MODULO DESPLIEGUE DE APLICACIONES WEB

UND 2

Practica 1 Configuración de Docker

Page | 6

